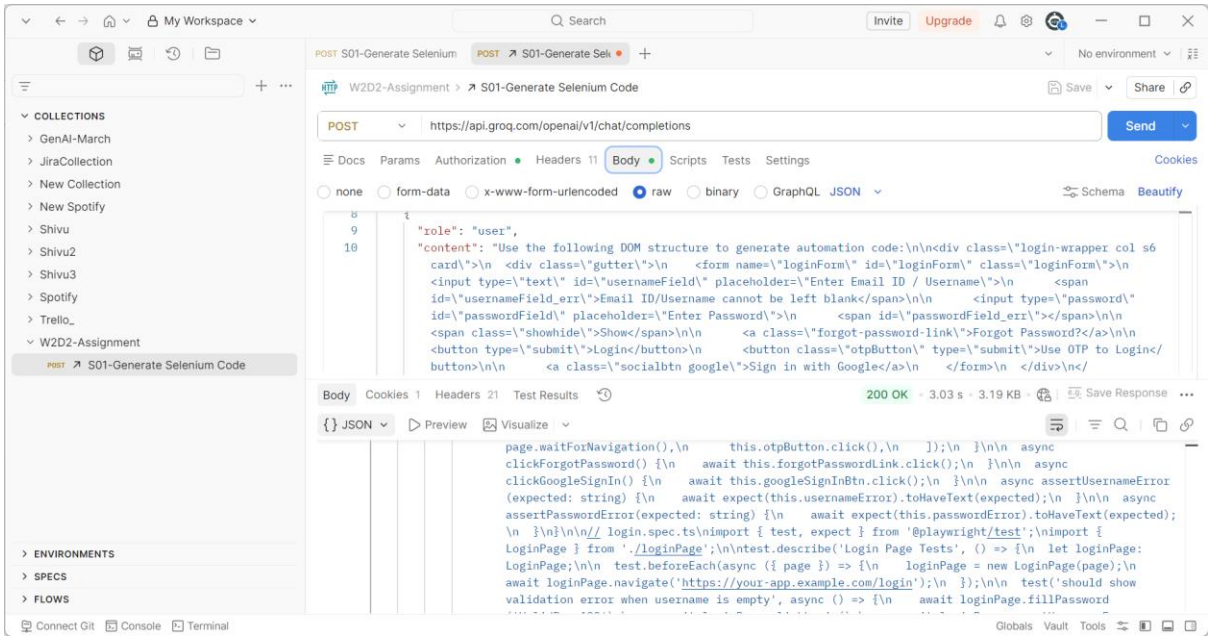
Generate Playwright-TS code




```
// loginPage.ts
import { Page, Locator, expect } from '@playwright/test';


export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly usernameError: Locator;
  readonly passwordInput: Locator;
  readonly passwordError: Locator;
  readonly showHideToggle: Locator;
  readonly forgotPasswordLink: Locator;
  readonly loginButton: Locator;
  readonly otpButton: Locator;
  readonly googleSignInBtn: Locator;


  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.locator('input[placeholder="Enter Email ID / Username"]');
```

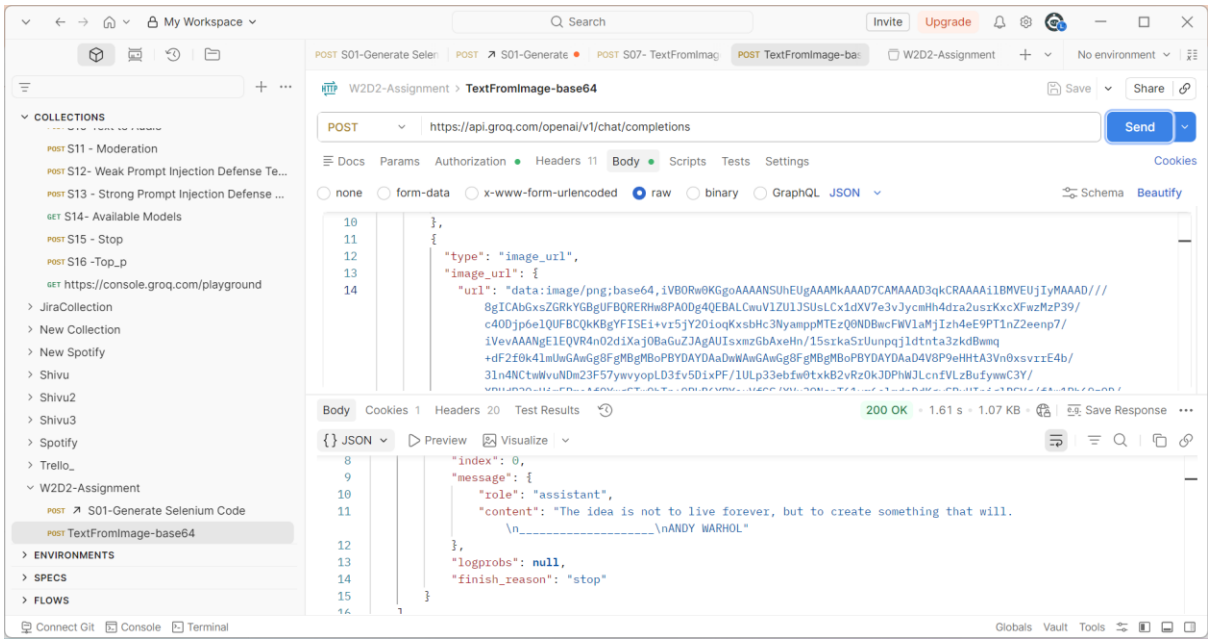
```
  this.usernameError = page.locator('#usernameField_err');
  this.passwordInput = page.locator('input[placeholder="Enter Password"]');
  this.passwordError = page.locator('#passwordField_err');
  this.showHideToggle = page.locator('span.showhide', { hasText: 'Show' });
  this.forgotPasswordLink = page.locator('a.forgot-password-link');
  this.loginButton = page.locator('button', { hasText: 'Login' });
  this.otpButton = page.locator('button.otpButton');
  this.googleSignInBtn = page.locator('a.socialbtn.google');
}

async navigate(url: string) {
  await this.page.goto(url);
  await expect(this.usernameInput).toBeVisible();
}

async fillUsername(username: string) {
  await this.usernameInput.fill(username);
}

async fillPassword(password: string) {
  await this.passwordInput.fill(password);
}

async clickShowHidePassword() {
  await this.showHideToggle.click();
}

async clickLogin() {
  await Promise.all([
    this.page.waitForNavigation(),
    this.loginButton.click()
```

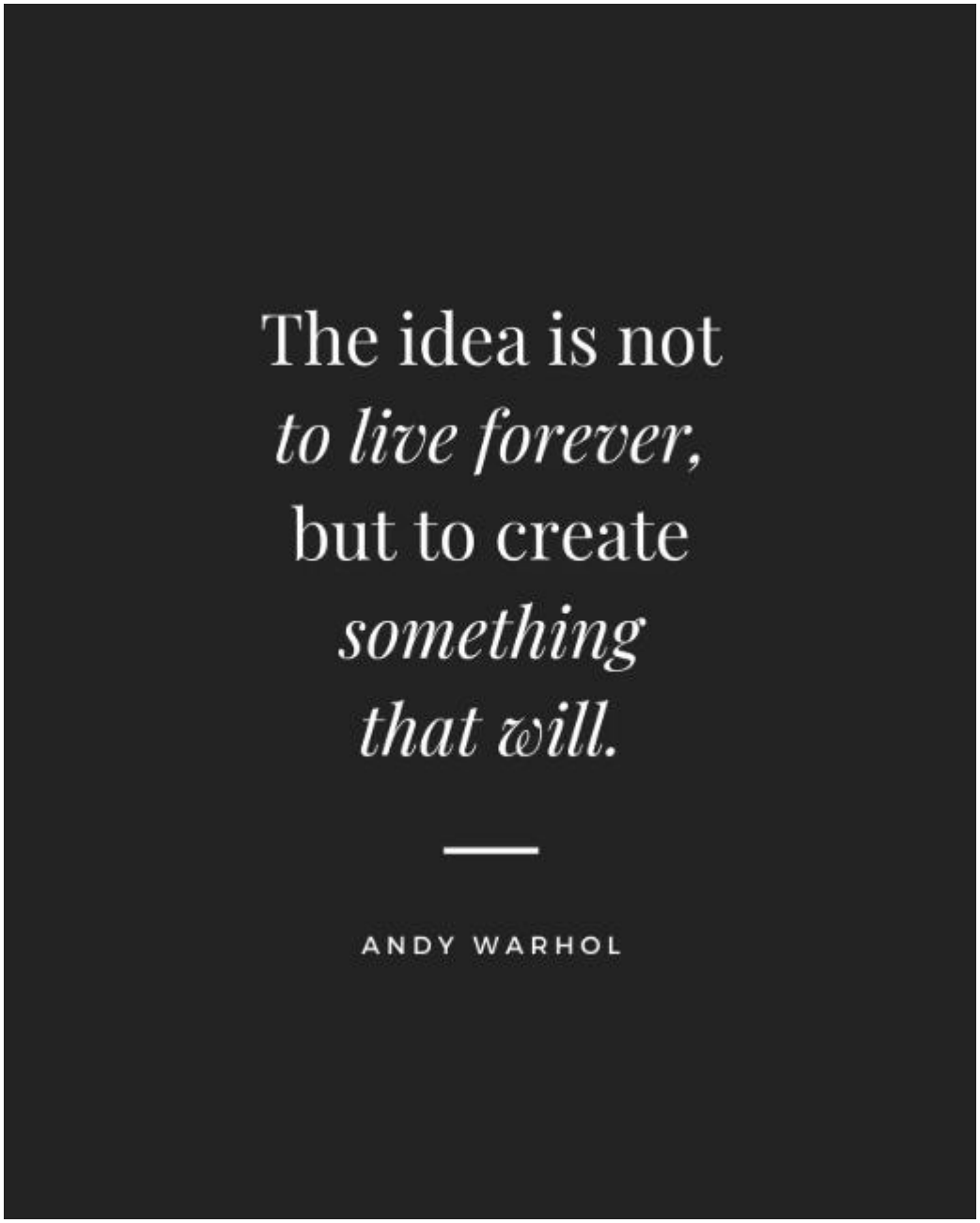
```
    ]);
  }

  async clickUseOtp() {
    await Promise.all([
      this.page.waitForNavigation(),
      this.otpButton.click()
    ]);
  }

  async clickForgotPassword() {
    await this.forgotPasswordLink.click();
  }

  async clickGoogleSignIn() {
    await this.googleSignInBtn.click();
  }

  async assertUsernameError(expected: string) {
    await expect(this.usernameError).toHaveText(expected);
  }

  async assertPasswordError(expected: string) {
    await expect(this.passwordError).toHaveText(expected);
  }
}

// login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from './loginPage';
```

```
test.describe('Login Page Tests', () => {
  let loginPage: LoginPage;

  test.beforeEach(async ({ page }) => {
    loginPage = new LoginPage(page);
    await loginPage.navigate('https://your-app.example.com/login');
  });

  test('validate username required error', async () => {
    await loginPage.fillPassword('ValidPass123');
    await loginPage.clickLogin();
    await loginPage.assertUsernameError('Email ID/Username cannot be left blank');
  });

  test('validate successful login', async ({ page }) => {
    await loginPage.fillUsername('user@example.com');
    await loginPage.fillPassword('ValidPass123');
    await loginPage.clickLogin();
    await expect(page).toHaveURL(/dashboard/);
  });

  test('validate password visibility toggle', async () => {
    await loginPage.fillPassword('Secret');
    const before = await loginPage.passwordInput.getAttribute('type');
    await loginPage.clickShowHidePassword();
    const after = await loginPage.passwordInput.getAttribute('type');
    expect(before).toBe('password');
    expect(after).toBe('text');
  });

  test('validate forgot password navigation', async () => {
```

```
  await loginPage.clickForgotPassword();
  await expect(loginPage.page).toHaveURL(/forgot-password/);
 });

 test('validate google sign-in flow', async () => {
  const [popup] = await Promise.all([
   loginPage.page.waitForEvent('popup'),
   loginPage.clickGoogleSignIn()
  ]);
  await expect(popup).toHaveURL(/accounts\\.google\\.com/);
 });
});
```

Text from image


My Workspace
Search
Invite
Upgrade
W2D2-Assignment
TextFromImage-base64
POST
https://api.groq.com/openai/v1/chat/completions
Send
Docs
Params
Authorization
Headers 11
Body
Scripts
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
"type": "image_url",
"image_url": {
"url": "
8gICAbGxsZGRkYGBgUFBQRERHw8PAODg4QEBALCwuVlZUlJSUsLCx1dXV7e3vJycmHh4dra2usrKxcXFwzMzP39/
c4ODjp6elQUFBCQkKBgYFISEi+vr5jY2ioqKxsbHc3NyamppMTEzQ0NDBwcFWVlaMjIzh4eE9PT1nZ2eenp7/
iVevAAANgElEQVR4nO2diXaj0BaGuZZAsxmZbAxeHn/15srkrBwmq
+dF2f0k41mUwGAwGg8BgPBgPBgMBgPBgMBgPBgM



{
"id": "chatcmpl-5908cde5-e559-4232-9b99-77401a3af7b5",
"object": "chat.completion",
"created": 1776920621,
"model": "meta-llama/llama-4-scout-17b-16e-instruct",
"choices": [
{
"index": 0,
"message": {

"role": "assistant",

"content": "The idea is not to live forever, but to create something that will.\n_____\nANDY WARHOL"

},

"logprobs": null,

"finish_reason": "stop"

}

],

"usage": {

"queue_time": 0.063014149,

"prompt_tokens": 177,

"prompt_time": 0.007319251,

"completion_tokens": 23,

"completion_time": 0.052007396,

"total_tokens": 200,

"total_time": 0.059326647

},

"usage_breakdown": null,

"system_fingerprint": "fp_37da608fc1",

"x_groq": {

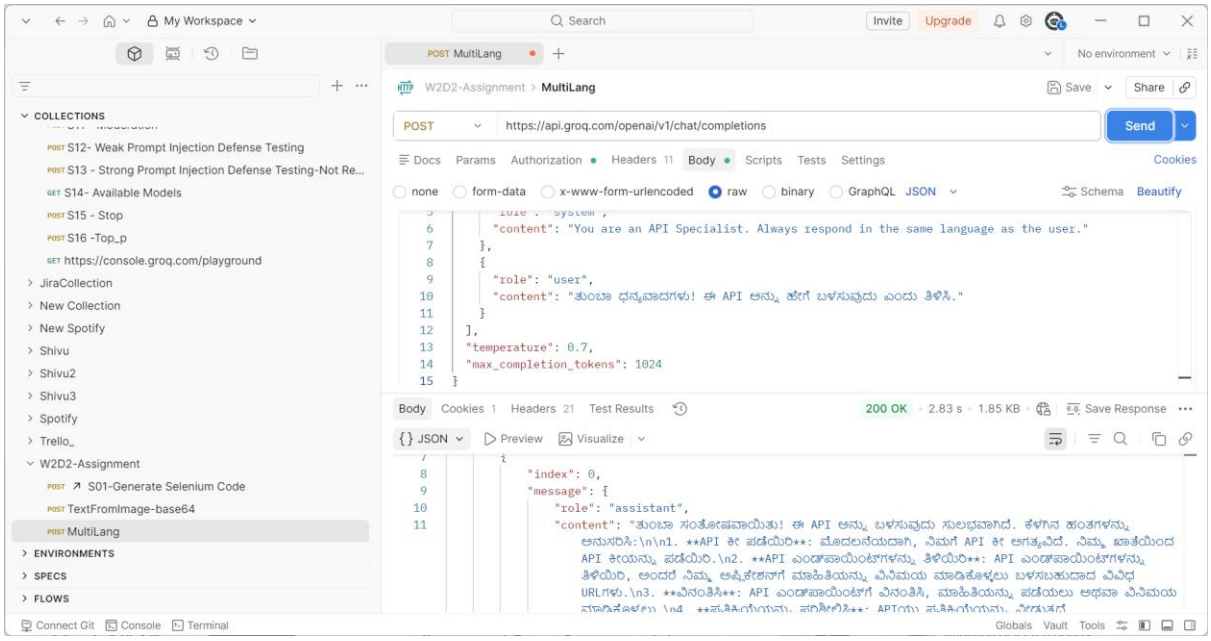
"id": "req_01kpwbhw1df8ptqsd168tppkvd",

"seed": 1196448312

},

"service_tier": "on_demand"

}



{

"id": "chatcmpl-0abe2f40-0b4d-4a69-b1ce-178c58353c3e",

"object": "chat.completion",

"created": 1776922155,

"model": "llama-3.3-70b-versatile",

"choices": [

{

"index": 0,

"message": {

"role": "assistant",

"content": "ತುಂಬಾ ಸಂತೋಷವಾಯಿತು! ಈ API ಅನ್ನು ಬಳಸುವುದು ಸುಲಭವಾಗಿದೆ. ಕೆಳಗಿನ ಹಂತಗಳನ್ನು ಅನುಸರಿಸಿ:\n\n1. **API ಕೀ ಪಡೆಯಿರಿ**: ಮೊದಲನೆಯದಾಗಿ, ನಿಮಗೆ API ಕೀ ಅಗತ್ಯವಿದೆ. ನಿಮ್ಮ ಖಾತೆಯಿಂದ API ಕೀಯನ್ನು ಪಡೆಯಿರಿ.\n2. **API ಎಂಡ್‌ಪಾಯಿಂಟ್‌ಗಳನ್ನು ತಿಳಿಯಿರಿ**: API ಎಂಡ್‌ಪಾಯಿಂಟ್‌ಗಳನ್ನು ತಿಳಿಯಿರಿ, ಅಂದರೆ ನಿಮ್ಮ ಅಪ್ಲಿಕೇಶನ್‌ಗೆ ಮಾಹಿತಿಯನ್ನು ವಿನಿಮಯ ಮಾಡಿಕೊಳ್ಳಲು ಬಳಸಬಹುದಾದ ವಿವಿಧ URLಗಳು.\n3. **ವಿನಂತಿಸಿ**: API ಎಂಡ್‌ಪಾಯಿಂಟ್‌ಗೆ ವಿನಂತಿಸಿ, ಮಾಹಿತಿಯನ್ನು ಪಡೆಯಲು ಅಥವಾ ವಿನಿಮಯ ಮಾಡಿಕೊಳ್ಳಲು.\n4. **ಪ್ರತಿಕ್ರಿಯೆಯನ್ನು ಪರಿಶೀಲಿಸಿ**: APIಯು ಪ್ರತಿಕ್ರಿಯೆಯನ್ನು ನೀಡುತ್ತದೆ, ಅದನ್ನು ನಿಮ್ಮ ಅಪ್ಲಿಕೇಶನ್‌ಗೆ ಬಳಸಬಹುದು.\n\nಉದಾಹರಣೆಗೆ, ನಿಮ್ಮ ಅಪ್ಲಿಕೇಶನ್‌ಗೆ ಬಳಸಲು ಬಳಕೆದಾರರ ಮಾಹಿತಿಯನ್ನು ಪಡೆಯಲು, ನಿಮ್ಮಗೆ `https://api.example.com/users` ಎಂಡ್‌ಪಾಯಿಂಟ್ ಇರಬಹುದು. ನಿಮ್ಮಗೆ ಈ ಎ"

},

"logprobs": null,

```
            "finish_reason": "length"
        }
    ],
    "usage": {
        "queue_time": 0.160108186,
        "prompt_tokens": 140,
        "prompt_time": 0.014302881,
        "completion_tokens": 1024,
        "completion_time": 2.334469735,
        "total_tokens": 1164,
        "total_time": 2.3487726159999998
    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_45180df409",
    "x_groq": {
        "id": "req_01kpwd0kn4ey4rebfvbc87cs7b",
        "seed": 1411556186
    },
    "service_tier": "on_demand"
}
```